

SDÜ/Mühendislik Fakültesi/Bilgisayar Mühendisliği
Veri Yapıları ve Algoritmaları Vize 2021-22 Kasım

Soru 1. Aşağıdaki ifadelerin tanımını yapınız.

a) Soyut veri tipi (abstract data type)

CEVAP: Bir veri yapısının matematiksel veya mantıksal modelidir.

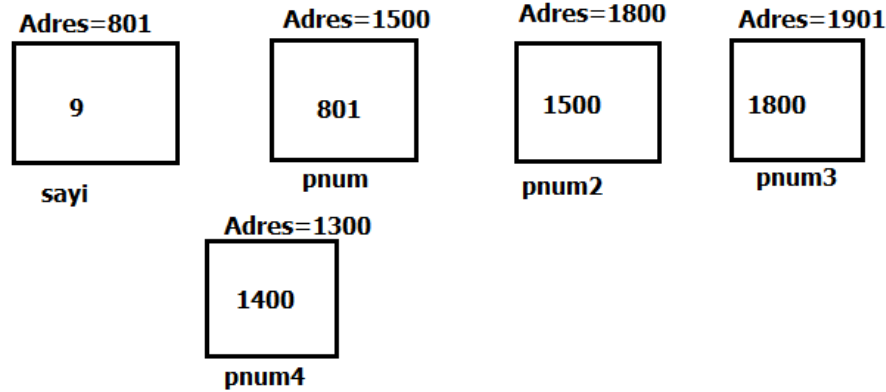
b) Veri tipi (data type)

CEVAP: Soyut veri tipinin uygulanmasıdır.

c) Veri yapısı (data structure)

CEVAP: Pointer yardımıyla birden fazla veri tipi üzerinde özel bir ilişki kurulmasıdır.

Soru 2. Aşağıda verilen şekilde sayı (int), pnum (pointer), pnum2 (pointer), pnum3 (pointer) ve pnum4 (pointer) tiplerini göstermektedir. Şekle ait gerekli ise kütüphane tanımlı yapın ve main fonksiyonu içinde kodları yazınız.



CEVAP:

```
#include <stdio.h>
int main()
{
    int sayi=9, *pnum;int *pnum4;
    pnum = &sayi;
    int **pnum2;
    pnum2=&pnum;
    int ***pnum3;
    pnum3=&pnum2;
    printf("%d",***pnum3);
    return 0;
}
```

Soru 3. Aşağıdaki fonksiyon tek yönlü bağlı listedeki ilk üç düğümün boyutu kadar düğümü liste sonuna eklemektedir. Bu fonksiyonun doğru çalışması için şimdiki durumda hatalı olan 3 satırın üzerine çizip yan tarafa doğrusunu yazınız.

```
void hesapla(struct node *head){
    struct node *tmp=head;struct node *tmp2;
    int adet;
    adet=sizeof(tmp);
    tmp=tmp->next;
    adet+=sizeof(tmp);
    tmp=tmp->next->next;
    adet+=sizeof(tmp);
    while(adet>1)
    {
        tmp2=(struct node*)malloc(sizeof(struct node));
        tmp=head;
        while(tmp->next!=NULL)
            tmp=tmp->next;
        tmp->next=tmp2;
        tmp2->data=1;
        adet=adet-1;
    }
    tmp=head;
    while(tmp!=NULL)
    {
        printf("%d\t",tmp->data);tmp=tmp->next;
    }
}
```

CEVAP:

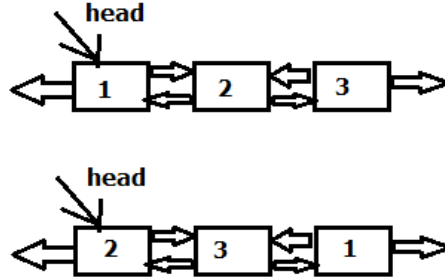
```
void hesapla(struct node *head){
    struct node *tmp=head;struct node *tmp2;
    int adet;
    adet=sizeof(tmp);
    tmp=tmp->next;
    adet+=sizeof(tmp);
    tmp=tmp->next;
    adet+=sizeof(tmp);
    while(adet>0)
    {
        tmp2=(struct node*)malloc(sizeof(struct node));
        tmp=head;
        while(tmp->next!=NULL)
            tmp=tmp->next;
        tmp->next=tmp2;
        tmp2->data=1;
        adet=adet-1;
    }
    tmp=head;
```

```

while(tmp!=NULL)
{
    printf("%d\t",tmp->data);tmp=tmp->next;
}
}

```

Soru 4. Bir çift yönlü bağlı listedeki ilk düğümü silip bu düğümün değerini sona yeni düğüm olarak ekleyen “hesapla” isimli fonksiyonu yazınız. (main yazmayınız hazır olduğunu farzedin)



CEVAP:

```

#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;struct node *next;struct node *prev;
};
void hesapla(struct node **head){
    struct node *tmp=(*head);
    int alinan=tmp->data;
    (*head)=tmp->next;
    free(tmp);
    struct node *yeni=(struct node*)malloc(sizeof(struct node));
    yeni->next=NULL;yeni->data=alinan;
    tmp=(*head);
    while(tmp->next!=NULL)
    tmp=tmp->next;
    tmp->next=yeni;yeni->prev=tmp;
}
void printList(struct node *head){
    struct node *tmp=head;
    while(tmp!=NULL)
    {
        printf("%d\t",tmp->data);tmp=tmp->next;
    }
}

```



```

int main()
{
    struct node *dugum1,*dugum2,*dugum3;
    dugum1=(struct node*)malloc(sizeof(struct node));
    dugum2=(struct node*)malloc(sizeof(struct node));
    dugum3=(struct node*)malloc(sizeof(struct node));
    dugum1->next=dugum2;dugum2->next=dugum3;dugum3->next=NULL;
    dugum1->prev=NULL; dugum2->prev=dugum1;dugum3->prev=dugum2;
    dugum1->data=1;dugum2->data=2;dugum3->data=3;
    hesapla(&dugum1);printList(dugum1);
    return 0;
}

```

Soru 5.

a. Yığın için kullanılan peek() ve pop() fonksiyonlarının farkını yazınız.

Cevap: ikisi de yığının en üstündeki elemanı alır ancak peek'te top eksilmez yani düğüm tamamen silinmediği için her peek çağrımında aynı eleman gösterilir.

b. Bir yığın içindeki tek sayıları silen “donustur” isimli fonksiyonu yazınız. (Geçici yığın oluşturabilirsiniz ancak aynı yığın üzerinde değişiklik tamamlanmalı)



```
// C program for array implementation of stack
```

```
#include <limits.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// A structure to represent a stack
```

```
struct stack {
```

```
    int top;
```

```
    unsigned capacity;
```

```
    int *array;
```

```
};
```

// function to create a stack of given capacity. It initializes size of
// stack as 0

```
struct stack *createstack(unsigned capacity)
{
    struct stack *stack = (struct stack*)malloc(sizeof(struct stack));
    stack->capacity = capacity;
    stack->top = -1;
    stack->array = (int*)malloc(stack->capacity * sizeof(int));
    return stack;
}
```

// Function to add an item to stack. It increases top by 1

```
void push(struct stack *stack, int item){
```

```
    stack->array[++stack->top] = item;
```

```
}
```

```
void donustur(struct stack *head){
```

```
    int tek=0;int deger;
```

```
    struct stack *gecici = createstack(100);
```

```
    while(head->top!=-1){
```

```
        deger=head->array[head->top--];
```

```
        if(deger%2==0)
```

```
            push(gecici,deger);
```

```
    }
```

```
    while(gecici->top!=-1){
```

```
        head->array[++head->top]=gecici->array[gecici->top--];
```

```
    }
```

```
}
```

tek array siler

// Driver program to test above functions

```
int main()
```

```
{
```

```
    struct stack *stack = createstack(100);
```

```
    push(stack, 6);
```

```
    push(stack, 5);
```

```
    push(stack, 8);
```

```
    push(stack, 4);
```

```
    donustur(stack);  
    printf(".....%d\n",stack->array[stack->top--]);  
    printf(".....%d\n",stack->array[stack->top--]);  
    printf(".....%d\n",stack->array[stack->top--]);  
    return 0;  
}
```